Full Claude Context - Liane

EEG Tokenizer — Full Project Claude Context Breakdown

Project: Stanford CS 231N — Multi-modal foundation model on 4M architecture
Status: through 2026-06-05
Repo: `neural-foundation-model` (EEG work in a fork)

This document is the complete context dump for the EEG tokenizer work. It supersedes nothing — `state.md` is the running session log, `CLAUDE.md` (root) is the project rulebook, `neural_tokenizers/CLAUDE.md` is the eval contract, `neural_tokenizers/meg/CLAUDE.md` is the MEG track. This doc collapses all of those plus session history into one self-contained file. Read this first if you are new.

0. TL;DR

The EEG track shipped a LaBraM `vqnsp.pth` finetune slug `V8192_d64_ch17_sr200_train-eeg1+2_e5`, checkpoint-5 of 10 epochs trained on combined THINGS-EEG1 + THINGS-EEG2 (60 subjects, 2,039,312 trials), producing 17 integer tokens per trial from a vocab of 8,192. The per-subject `.npz` caches were repacked into a unified 27-shard catalog-range format that 4M's standard loader pairs with RGB. As of 2026-06-05 the repack has been independently verified clean (0 violations, all 2.04M trial rows accounted for).

The §5 evaluation harness shows the tokenizer is healthy on three of four axes (codebook 83% utilized, sequence entropy gap 49%, near-perfect channel correlation) but the linear probe is at the random floor (0.55% top-1 vs 0.67% random, with a 3.25% raw-signal ceiling on 200-way THINGS image identity). Three A/B diagnostics (collapse, F3 window context, F2 preprocessing filter) all showed the inference-side recipe is not the limiter; the trial-averaging diagnostic showed the tokenizer is the limiter (it collapses on cleaner input). Synthesis: per-image linear identity decodability is bounded by the tokenizer's single-trial-noise overfit, not by EEG signal quality or by inference choices.

The preliminary 4M scaling result is that the biggest model performs *worse* when EEG (or MEG) is included than smaller models do. This is consistent with "tokens look like noise from a discrimination POV; larger models have more capacity to fit that noise."

---

1. Project context

1.1 Research question being pursued (narrowed scope)

Does adding EEG as a modality improve 4M's scaling behavior on image tasks?

The trunk lead trains mini-4M from scratch on RGB + depth, and again on RGB + depth + EEG. The control is the no-neural run. Evaluation: model loss on RGB and downstream readouts (e.g., ImageNet classification with a nonlinear probe).

1.4 Tokenization plan (decided)

- **RGB and CLIP**: reuse Apple's pretrained 4M tokenizers off-the-shelf.
  - `EPFL-VILAB/4M_tokenizers_rgb_16k_224-448`
  - `EPFL-VILAB/4M_tokenizers_CLIP-B16_8k_224-448`
  - No retraining.
- **EEG**: finetune LaBraM's pretrained `vqnsp.pth` (94.8 MB, real PyTorch checkpoint, pretrained on ~2,500 h of scalp EEG) on THINGS-EEG2 (primary) and THINGS-EEG1 (secondary). Target: 8k-codebook discrete EEG tokenizer producing integer tokens from channel-time patches.
- **MEG**: handled by the MEG track. Two-tokenizer approach: μ-transform (Cho2026 baseline) + BrainOmni BrainTokenizer (RVQ, finetuned). BrainOmni 3b shipped as the production MEG tokenizer.

1.5 Train/val split convention

EEG and MEG both use an **image-level 80/20 train/val split**. "Image-level" means: for each unique stimulus image, *all* trials of that image go entirely to train or entirely to val. An image's trials are never split between train and val.

The actual production split is `things_split.json` (`/project/data/things_split.json`):
- Policy: `image_level_val_from_neural_intersection`, seed=0.
- Train: 22,763 image_ids (the catalog minus val).
- Val: 3,344 image_ids (the strict intersection: catalog ∩ MEG ∩ EEG1 ∩ EEG2). This ensures val images have all four modalities.

Note: this overrides the earlier seed-20200220 80/20 split that was used for the tokenizer-training HDF5 (which is irrelevant to 4M training; it was only for evaluating LaBraM-finetune training loss).

---

2. The EEG data: THINGS-EEG1 vs THINGS-EEG2

2.1 THINGS-EEG2 (Gifford et al. 2022)

- **Source**: OSF project `3jk45`, preprocessed component GUID `anp5v`. HuggingFace mirrors also exist.
- **On-volume path**: `/project/data/raw/things-eeg2/sub-XX/preprocessed_eeg_{training,test}.npy`
- **Files**: per subject, a numpy dict (loaded via `np.load(...).item()`) with key `preprocessed_eeg_data`.
- **Shapes**:
  - training: `(16540, 4, 17, 100)` = (images, reps, channels, timepoints)
  - test: `(200, 80, 17, 100)`

- **Channels**: 17 occipital + posterior, regex-selected as `^O *|^P *` from the original 63 by Gifford's preprocessing. Names (verified in conversion script): `P1, P2, P3, P4, P5, P6, P7, P8, PZ, PO3, PO4, PO7, PO8, POZ, O1, O2, OZ`
- **Sampling rate**: 100 Hz.
- **Scale**: **MVNN-whitened** (multivariate noise normalization), zero-mean unit-variance per channel, dimensionless — NOT µV. This is the source of the calibration story in §6.
- **Epoch window**: −200 ms to +800 ms (so 100 timepoints at 100 Hz = 1 s total).
- **Subjects**: sub-01..sub-10 (10 total).
- **Paradigm**: slow event-related, proper ISI. Per-trial SNR is relatively clean.
- **Total trials/subject**: 16,540 × 4 + 200 × 80 = 82,160.

2.2 THINGS-EEG1 (Grootswagers et al. 2022)

- **Source**: OpenNeuro `ds003825`.
- **On-volume path**: `/project/data/raw/things-eeg1/derivatives/eeglab/sub-XX_task-rsvp_continuous.set` (EEGLAB .set + .fdt) plus `/project/data/raw/things-eeg1/bids_events/sub-XX/eeg/*.tsv` for stim onset codes.
- **Source format**: EEGLAB `.set` / `.fdt` in `derivatives/`. NOT `.npy`. We pulled only `derivatives/` + `bids_events/` to avoid TB-scale raw.
- **Sampling rate**: 250 Hz raw → resampled to 200 Hz in our conversion.
- **Scale**: µV (MNE reads V; we ×1e6 to µV).
- **Channels**: same 17 posterior/occipital subset, selected by name match against EEGLAB channel labels.
- **Epoch window**: −200 ms to +800 ms post-stim, extracted per E1 stimulus onset event.
- **Subjects**: sub-01..sub-50 (50 total).
- **Paradigm**: RSVP (rapid serial visual presentation) — each image shown briefly, one rep per (image, subject). Per-trial SNR is noisier than EEG2.
- **Total trials/subject**: ~22,448 (varies slightly).
- **Subject-level artifact problem**: ~18 of 50 subjects have channels with std up to 3,990 µV (artifact contamination). This is why our calibration target uses the *median* across EEG1 subjects, not the mean.

2.3 EEG1 vs EEG2 side by side

dimension	EEG1	EEG2
Subjects	50	10
Reps per (image, subject)	1 (RSVP single-shot)	4 train / 80 test
Images covered	22,448	16,740
Paradigm	RSVP (fast, brief)	Slow event-related (proper ISI)
Native sfreq	250 Hz	100 Hz
Scale	µV	MVNN-whitened
Calibration	None needed	Per-subject std → EEG1 median µV
Source format	EEGLAB .set/.fdt + BIDS events	numpy dict
Subject-level artifacts	~18/50 noisy	Clean
Total trials (pool)	~1.12M	~0.82M

2.4 Coverage of the THINGS catalog (26,107 images)

- EEG1 covers: **22,448** (`image_ids_eeg1` in `eeg_coverage.json`)
- EEG2 covers: **16,740**
- Union: **22,470**
- Intersection: **16,718**
- EEG2-only: 22 images
- Uncovered (no EEG): 3,637

`eeg_coverage.json` at `/project/data/eeg_coverage.json` is the source of truth for this mapping; it was built by `modal_eeg_write_coverage_json.py` and joins on `things_catalog.json` filenames.

---

3. Decided tokenizer approach: LaBraM Stage 2c

`neural_tokenizers/CLAUDE.md` §4 lays out three Stage 2 options for EEG:

	Our own scratch design	**LaBraM's architecture**
From scratch weights	(2a) Generic, modality-agnostic	(2b) Borrow design, not data
Pretrained + finetuned	—	**(2c) Borrow design AND data**

We chose **2c**: load LaBraM's pretrained `vqnsp.pth` weights and finetune on THINGS-EEG. This is the only path that exploits the 2,500+ hours of EEG LaBraM saw in pretraining. EEG-only: MEG and TVSD still need 2a or 2b, so committing to 2c means maintaining two codepaths long-term, which we accepted.

The decision blocker was supposed to be Stage 1 baseline numbers on the §5 harness before ranking 2a/2b/2c. We did build the Stage 1 baseline (`eeg_tokenizer.py::Stage1EEGTokenizer`, BottleneckMLP + memcodes-style quantizer) but ended up going straight to 2c because LaBraM's pretrained weights were

available and the milestone clock was running.

---

4. LaBraM technical details (verified end-to-end)

Verified 2026-05-22 by reading the LaBraM repo at `https://github.com/935963004/LaBraM` (now pinned as a submodule at `external/LaBraM`). Cite line numbers when changing behavior — they're the source of truth, not the prose below.

4.1 vqnsp.pth contents

- 94,805,201 bytes (94.8 MB).
- Top-level keys: `model`, `optimizer`, `epoch=99`, `scaler`, `args`.
- `model` contains: 12-layer encoder NeuralTransformer with `TemporalConv` patch embed, 3-layer decoder NeuralTransformer with `PatchEmbed.proj`, NormEMAVectorQuantizer (`quantize.embedding.weight` + EMA tracking buffers), amplitude/phase reconstruction heads (`encode_task_layer.*`, `decode_task_layer.*`, `decode_task_layer_angle.*`).
- Saved args: `codebook_n_emd=8192, codebook_emd_dim=64, input_size=1600, batch_size=128, quantize_kmeans_init=True`.

4.2 Codebook structure

- Vocab: **8,192**
- Embedding dim: **64** (NOT the CLI default of 32 — must override)
- K-means initialized, EMA-updated
- L2-normalized at registration

4.3 Stage-1 loss (verified at `modeling_vqnsp.py:149-175`)

Per patch:
1. FFT along time (200 samples).
2. `std_norm` separately on amplitude and phase spectra, across `(N, A, T)` within the batch.
3. Decode the quantized code back to amplitude and phase.
4. Loss = `emb_loss + MSE(amplitude_hat, amplitude) + MSE(phase_hat, phase)`, equal 1:1:1 weighting.

Crucial implication: the loss never sees absolute µV scale, only spectral shape. The `/100` constant divisor in `engine_for_vqnsp.py:57` is washed out by `std_norm`.

4.4 HDF5 input format (`dataset_maker/make_h5dataset_for_pretrain.py`)

```
<file>.hdf5
└── <recording_stem>/                  # one HDF5 group per subject
    └── eeg
        ├── data: float64, shape (C, T)  # CONTINUOUS (not trial-epoched)
        ├── chunks=(C, rsFreq)           # e.g. (17, 200)
        └── attrs: lFreq, hFreq, rsFreq, chOrder (uppercase channel names)
```

The dataset is one long `(C, T_total)` array per subject. `SingleShockDataset` slides a window over it. To feed trial-structured data we concatenate trials end-to-end and set `stride_size = window_size = 200` if we wanted 1-trial windows. (We actually use windows of 1600 = 8 trials; see §5.)

4.5 Channel-name constraint

Every channel name must appear in LaBraM's hardcoded 10-20 list at `utils.py:42-57` (`standard_1020`). Missing names raise `ValueError` from `utils.py:713-717` (`standard_1020.index(ch_name) + 1`). All 17 THINGS-EEG2 posterior channels are present. There is no minimum count enforced — the position embedding has 129 slots, so any 1–128 channel subset works.

4.6 Sampling rate constraint

`SingleShockDataset.__getitem__` does raw indexing only; no resampling at load time. `window_size = time_window * 200` is hardcoded. **The HDF5 must be at 200 Hz at write time.** We upsample EEG2 from 100→200 with `scipy.signal.resample_poly(up=2, down=1)` and resample EEG1 from 250→200 with `up=4, down=5`.

4.7 Filter conventions (per README + `dataset_maker/shock/utils/eegUtils.py`)

- 0.1–75 Hz FIR bandpass
- 50 Hz notch
- Resample to 200 Hz
- Units = µV (per LaBraM docs, but see §6 for what we actually do)

4.8 CLI quirks

- **No `-data_path` flag.** HDF5 paths are hardcoded at `run_vqnsp_training.py:149-162` (`datasets_train`, `datasets_val`, `time_window`). We patch the script in the Modal wrapper at launch time; we do not commit edits to the submodule.
- **`torchrun` is required.** Even on a single GPU, the script always calls `init_distributed_mode` and builds a `DistributedSampler`. Launch via `torchrun --nproc_per_node=1 run_vqnsp_training.py ...`.
- **`-resume vqnsp.pth` also loads optimizer + scaler** and hard-sets `start_epoch=1` (`utils.py:610-647`). For a clean finetune LR schedule, strip `optimizer` and `scaler` from the checkpoint before resuming. We have a helper that does this and writes `vqnsp_model_only.pth` once per run.
- **CLI overrides we need** to match the shipped `vqnsp.pth`: `-codebook_n_emd 8192 --codebook_emd_dim 64 --quantize_kmeans_init`.

---

5. The decoder shape bug and what shipped

5.1 The bug

Per the spec and LaBraM's own README, the natural way to train on 1-second trials would be `--input_size 200` with `time_window=1` (one 1-second patch per channel). When we tried this, the model's decoder hit a shape error.

What goes wrong: with `time_window=1`, the `quantize` output tensor is `(B, D, 17, 1)` where the last dim is the time-patch axis with size 1. The decoder checks `t == patch_size`; here `t=1` and `patch_size=1` happen to match, so the branch that handles single-patch inputs fires. That branch misreads the time-window axis (`a=17` interpreted as time, instead of `t=1`), expanding `pos_embed` to `17*17+1 = 290` tokens vs the 18 actual tokens. The training pipeline crashes or produces nonsense.

5.2 The shipped workaround: `-input_size 1600`, `time_window=8`

We use the pretrained `vqnsp.pth`'s native window size (1600 samples = 8 patches × 200), which is what the decoder was trained with. The HDF5 holds a continuous concatenation of 1-second trials per subject, and a training window of 1600 samples covers 8 consecutive trials.

At inference time, we get a single 1-second trial to tokenize. We tile it into 8 identical patches to match the model's native sequence length `(B, 17, 8, 200)`, call `get_codebook_indices`, and take patch 0 of each channel via `tokens_all[:, ::8]` to recover the 17 per-channel codes.

Cited files:
- Workaround: `neural_tokenizers/eeg/modal/finetune_labram_tokenizer.py:118-123,313`
- Inference tiling: `neural_tokenizers/eeg/labram_tokenizer.py:78-85`

5.3 The consequence: a train/inference window-context mismatch

Training windows = 8 distinct consecutive trials.
Inference windows = 8 identical copies of one trial.

This is the **F3 concern** that the code review surfaced. The encoder's attention is bidirectional across the 8 patches, so what sits in the other 7 patches affects patch 0's code. We empirically tested this (§14.2): swapping 8-copies for 8-real-neighbors flips ~49% of per-channel codes but does not move the linear probe (both arms at the random floor). So F3 is real but not actionable.

5.4 The collapse: 136 → 17

`get_codebook_indices` returns `(B, 136)` = 17 channels × 8 time-patches, channel-major order `[ch0_t0..ch0_t7, ch1_t0..]`. We slice `[:, ::8]` to keep patch 0 of each channel → 17 codes.

This was originally documented (incorrectly) as "lossless dedup because all 8 patch codes are identical for tiled input." Measurement on 512 trials (§14.1) showed only ~80% of (trial, channel) pairs have all 8 patches identical; patch 0 is the majority code 93% of the time. Comment in `labram_tokenizer.py` corrected to reflect measurement.

---

6. Calibration journey (the MVNN → µV story)

6.1 The worry

Gifford's preprocessed EEG2 is MVNN-whitened (zero-mean unit-var per channel, dimensionless). LaBraM's pretraining expected µV. Naively that's a scale mismatch the model wouldn't tolerate.

6.2 The spectral `std_norm` analysis

LaBraM's loss runs FFT on each 200-sample patch and applies `std_norm` separately on amplitude and phase across `(N, A, T)` within the batch (`modeling_vqnsp.py:143-147, 157, 159`). The reconstruction MSE is computed against the std-normalized FFT, not against the raw signal. So:
- The `/100` constant divisor in `engine_for_vqnsp.py:57` is washed out by `std_norm`.
- MVNN's per-channel rescale doesn't change spectral shape (linear scalar).
- MVNN's cross-channel decorrelation is invisible to the per-channel tokenizer (each patch is one channel × 1 s, processed independently).

Conclusion at that time (2026-05-23): the loss should be scale-insensitive. **Default plan: run with MVNN-whitened data as-is, no rescale, no re-preprocessing.** Escalation paths documented:
1. (Default) MVNN as-is.
2. Empirical per-channel rescale to ≈10 µV std.
3. Re-preprocess from raw BIDS (~1 day; only if 1+2 both fail).

6.3 What actually happened: EEG2-only finetune codebook collapse

Running the smoke test (1 epoch on sub-01 EEG2 MVNN data) revealed near-total codebook collapse — only ~4 of 8,192 codes in use. The `std_norm` argument turned out to under-protect: while the loss is scale-invariant in the limit, the *intermediate activations* of the 12-layer encoder are sensitive to input distribution, and the pretrained codebook is calibrated to µV-range distributions. The encoder needed input in the same approximate range as pretraining.

6.4 Per-channel µV calibration

Resolution: rescale each EEG2 channel to roughly match EEG1's per-channel µV distribution. Concretely, per subject:

```
eeg2_ch_std = all_trials.std(axis=(0, 2), keepdims=True).clip(min=1e-6)   # (1, 17, 1)
all_trials *= eeg1_ch_std / eeg2_ch_std
```

Where `eeg1_ch_std` is a fixed 17-element vector loaded from `/project/data/things-eeg/labels/eeg1_per_channel_std.json`. This vector is **per-channel** (not a single global scalar) because occipital channels legitimately have higher variance than parietal channels, and the pretrained codebook expects that structure.

6.5 Mean vs median (artifact subjects)

The EEG1 per-channel std vector was initially built by averaging per-channel stds across all 50 EEG1 subjects. The result was a mean of 50.8 µV across channels — implausibly large for clean EEG. Investigation: ~18 of 50 EEG1 subjects have channels with per-channel std up to **3,990 µV** (artifact contamination, likely electrode pops or movement). These artifact-contaminated subjects dominated the mean.

Fix: **pool by median, not mean.** Median per-channel std across the 50 EEG1 subjects is 15.25 µV (matches clean-EEG expectations). The final calibration vector is the median.

This vector is **frozen** and applied byte-identically at training (`convert_eeg_to_labram_hdf5.py:206-217`) and at token-production time (`modal_eeg_produce_tokens.py:298-299`).

6.6 The F2 filter mismatch (an asymmetry that survived to shipping)

The training-time conversion (`convert_eeg_to_labram_hdf5.py:_filter_and_upsample`) applies a 0.1–75 Hz bandpass and 50 Hz notch at 200 Hz after upsampling. The production-time extractor (`modal_eeg_produce_tokens.py:_extract_eeg2_trials`) calibrates and upsamples but did NOT apply the filter — only the EEG1 production path filtered.

This was caught in the 2026-05-28 code review (F2). The code was fixed (added the filter to `_extract_eeg2_trials` with a loud DO-NOT-RE-RUN comment), then the eval A/B ran to decide whether to re-ship. Result (§14.3): no meaningful difference on any §5 axis. **Decision: do NOT re-ship.** Shipped (unfiltered) EEG2 tokens stay; the code fix is in place so any future token run matches training.

---

7. Training journey

7.1 The Stage 1 baseline (built but not shipped)

Per the §4 mandate ("get clean numbers across all four §5 axes for this baseline before touching anything"), we built:
- `neural_tokenizers/eeg/eeg_encoder.py` — `BottleneckMLPEncoder` (6-layer MLP on flattened `(17*100,)`)
- `neural_tokenizers/eeg/eeg_quantizer.py` — `ProductVQQuantizer` (8 codebooks × 1024 codes, straight-through gradient)
- `neural_tokenizers/eeg/eeg_decoder.py` — symmetric MLP decoder
- `neural_tokenizers/eeg/eeg_tokenizer.py` — `Stage1EEGTokenizer` composing the three

Trained 20 epochs on Modal (see `train_stage1.py`). Checkpoint at `/project/checkpoints/eeg/stage1/baseline/checkpoint.pt`. Used as the Stage-1 baseline number in §5 eval comparisons. Stays archived; not shipped to 4M.

7.2 1-epoch dry run

Per the spec: first Modal training run is `--epochs 1` on 1 subject's HDF5 to validate scale, codebook usage, and finite loss before committing GPU hours.

Watched for `Unused_code` log line from `engine_for_vqnsp.py:117-126`:
- Near 0 → codebook in use, scale OK.
- Near 8,192 → scale mismatch.

Initial dry run on uncalibrated MVNN-whitened sub-01 EEG2: `Unused_code` ~8,188 of 8,192 (collapse). After applying per-channel µV calibration: `Unused_code` ~1,100 of 8,192 (healthy). Proceeded to full run.

7.3 First archived run: 20-epoch EEG2-only

Earlier in the project: 20 epochs of finetune on EEG2 only (10 subjects). Reported a 5.5% top-1 probe number. Looked promising. But:
- That 5.5% was on a 100-class probe (random = 1%).
- Apples-to-apples re-eval on the 200-class probe (random = 0.67%) puts that same run essentially equivalent to the current shipped 10-epoch combined run.
- The "regression from 5.5% to 0.55%" was a probe-setup artifact (different class count), not a real regression.

This archived run lives at `/project/archive/eeg/labram_collapsed_run/`. Useful as an ablation reference; not shipped.

7.4 Shipped run: 10-epoch combined EEG1+EEG2 (checkpoint-5)

The production tokenizer. Setup:
- `finetune_labram_tokenizer.py::run_full_combined`
- 10 EEG2 subjects + 50 EEG1 subjects = 60 subjects
- `--input_size 1600 --codebook_n_emd 8192 --codebook_emd_dim 64 --quantize_kmeans_init --batch_size 64 --epochs 10 --warmup_epochs 3 --lr 5e-5 --num_workers 4`
- L40S GPU
- Per-channel µV calibration applied (§6.4–§6.5)
- Resume from `vqnsp_model_only.pth` (optimizer + scaler stripped)

Val loss bottomed at **epoch 5**, overfit after. So **checkpoint-5** is the production checkpoint, NOT the final-epoch checkpoint.

Slug: `V8192_d64_ch17_sr200_train-eeg1+2_e5`. Encodes: vocab 8192, embed dim 64, 17 channels, sampling rate 200, training set = EEG1+2, epoch 5.

Path: `/project/checkpoints/eeg/labram/V8192_d64_ch17_sr200_train-eeg1+2_e5/checkpoint.pt`

7.5 The trial-averaging diagnostic (the most important finding)

Run as a diagnostic on the shipped no-avg tokenizer. Question: "Does feeding cleaner input help?"

Setup: average 80 test reps per image (per subject) before tokenizing, then run §5 eval.

Result:
- Raw signal probe: 3.25% → 6.50% (doubles; cleaner input is more decodable).
- Codebook utilization: 83% → **40%** (codebook collapses on cleaner input).
- Token probe: 0.55% → **0.25%** (gets WORSE).

Interpretation: the tokenizer is overfit to the single-trial noise distribution. When given input it never saw during training (cleaner / higher SNR), it produces degenerate codes. This is strong evidence that the tokenizer is the limit, not the EEG signal. The raw signal has more information than the tokenizer is currently passing through.

---

8. Token production

8.1 The script: `modal_eeg_produce_tokens.py`

Runs the finetuned checkpoint over every EEG1 and EEG2 trial. Per-subject Modal function, parallelized across 60 subjects with `.spawn()`.

Key flow:
- `_extract_eeg2_trials`: load `.npy`, calibrate per-channel, upsample 100→200 (FILTER currently missing — F2 issue, code fix in place but unrun).
- `_extract_eeg1_trials`: load `.set/.fdt`, read E1 onset events from BIDS TSVs, epoch −0.2 to +0.8 s, filter 0.1–75 + notch 50 at 250 Hz, resample to 200.
- `_tokenize_trials_200hz`: tile each trial to 8 identical patches, call `model.get_codebook_indices`, take `[:, ::8]`.
- Map filenames (`aardvark_01b.jpg`) to 9-digit catalog `image_id` via `things_catalog.json`.
- Write `<source>_<subject>.npz` per subject.

Startup safety check: `_assert_eeg2_calibration_matches_hdf5` recomputes one trial via the production path and asserts it matches the stored training HDF5 within a 10% per-channel std tolerance. Currently passes (the missing-filter F2 difference is small enough to slip under 10%).

8.2 .npz cache format

Per file (e.g. `eeg1_sub-01.npz`):
- `tokens`: `(N, 17) int16`, values in `[0, 8192)`
- `image_id`: `(N,) <U9`, 9-digit zero-padded catalog ID
- `trial_idx`: `(N,) int16`, 0-indexed occurrence within (subject, source, image)
- `source`: scalar `<U4`, “eeg1” or “eeg2”
- `subject`: scalar `<U6`, “sub-01” etc.

Where: `/project/data/things-eeg/tokens/labram/V8192_d64_ch17_sr200_train-eeg1+2_e5/<source>_<subject>.npz`

60 files total (10 EEG2 + 50 EEG1).

8.3 The 2,039,312 number

Total trials across all 60 subjects. Used as the integrity-check number in the unified-shard verifier (§15).

8.4 Token production decisions

- **Trials kept separate, not averaged.** Decision 2026-05-25: each (image, source, subject, rep) is its own entry, capped at 5 per (image, source) in the old packer. Gives 4M more samples. The cap-of-5 was the old packer; the unified repack stacks all available trials.
- **EEG1 and EEG2 combined into one `tok_eeg/` dataset.** Source is encoded in filename suffix in the old per-trial pack (now obsolete), or implicit in the stacked-trials layout of the unified pack.

---

9. Volume layout (production artifacts)

Volume: `project` (Modal). App: `neural-fm`. Workspace: `neural-fm-data`. Mounted in containers at `/project`.

```
/project/
  checkpoints/
    eeg/
      labram/
        V8192_d64_ch17_sr200_train-eeg1+2_e5/
          checkpoint.pt                    # SHIPPED tokenizer (10ep ckpt-5)
          config.json                      # calibration + ckpt SHA256
        pretrained/
          vqnsp_original.pth               # LaBraM's shipped 94.8 MB
          vqnsp_model_only.pth             # opt+scaler stripped
      stage1/
        baseline/
          checkpoint.pt                    # Stage-1 BottleneckMLP baseline
  archive/
    eeg/
      labram_collapsed_run/                # 20ep EEG2-only archived run
  data/
    things_catalog.json                    # 26,107 image_id ↔ filename
    things_split.json                      # train_image_ids (22,763) + val (3,344)
    eeg_coverage.json                      # EEG1/EEG2 catalog coverage map
    raw/
      things-eeg2/sub-XX/
        preprocessed_eeg_{training,test}.npy
        image_metadata.npy
      things-eeg1/
        derivatives/eeglab/sub-XX_task-rsvp_continuous.set + .fdt
        bids_events/sub-XX/eeg/*.tsv
    things-eeg/
      preprocessed/eeg2/sub-XX_{train,val}.hdf5   # LaBraM-input HDF5s
      labels/
        eeg1_per_channel_std.json           # median µV calibration target
        eeg2_image_metadata.npy             # filename ↔ image index for EEG2
      tokens/
        labram/V8192_d64_ch17_sr200_train-eeg1+2_e5/
          config.json
          {eeg1,eeg2}_sub-XX.npz           # 60 per-subject token caches
    train/things/
      tok_rgb/shard_NNN.tar
      tok_eeg/shard_NNN.tar
      eeg_mask/shard_NNN.tar
      tok_meg/shard_NNN.tar
      meg_mask/shard_NNN.tar
      ... (depth, crop_settings)
    val/things/
      (same structure, 27 shards)
```

Note: The unified shards under `train/things/tok_eeg/` and `val/things/tok_eeg/` are owned by the MEG/infra track. The original per-trial pack is obsolete. The verifier confirms the unified pack is byte-faithful to the source `.npz` caches (§15).

---

10. Shard packing: original pack → unified repack

10.1 The original 23-shard per-trial pack (obsolete)

Originally `modal_eeg_pack_shards.py` packed the `.npz` caches into 23 train + 4 val WebDataset shards using `things_split.json`, with one entry per trial named `<image_id>_<source>sub<XX>_t<rep>.eeg.npy`. Trial cap of 5 per (image, source) for size control.

This is now obsolete. The team converged on the unified format (§10.2). The packer is kept in the repo but marked DO-NOT-RUN with a banner (it would `rmtree` the live `tok_eeg/` dir if executed).

10.2 The team-wide unified format decision (2026-05-26)

Driver: 4M's `unified_datasets.py` zips image-aligned modalities (rgb/depth/tok_rgb) row-by-row 1:1 — strict same-image-id-same-order requirement within each shard. Multi-trial neural modalities don't naturally fit because a single image has N MEG/EEG trials of different counts per dataset.

Solution: put neural modalities in the same dataset zipped by `image_id`, with stacked-trial `.npy` files plus a separate `eeg_mask` / `meg_mask` folder. Random trial selection + masking handled in a train-script `AbstractTransform`, no 4M edits, no separate dataloaders.

10.3 The unified 27-shard catalog-range repack

Layout (verified clean by `verify_unified_shards.py`):

```
/project/data/{train,val}/things/{tok_rgb,tok_depth,tok_meg,tok_eeg,meg_mask,eeg_mask,crop_settings}/shard_NNN.tar
```

- **27 shards per split.** Catalog-range: shard_N owns image_ids in `[N*1000, (N+1)*1000)`.
- **Train shard_000 has 881 members + val shard_000 has 119 members = 1000 image_ids in [0,1000).** "Holes" within a shard: in the train shard, val image_ids are absent (and vice versa).
- **One `.npy` per `image_id`** inside each shard (NOT one per trial — that's the format change).
- **tok_eeg member shape**: `(n_trials, 17) int16`. `n_trials` varies per image (4–~90+ depending on EEG1/EEG2 coverage and rep count). Sentinel `(1, 17)` all `1` for catalog images with no EEG.
- **eeg_mask member shape**: `(1,) uint8`, value 0 or 1 per image_id. 1 = covered, 0 = sentinel.
- **Member filename**: `<image_id>.npy` (9-digit zero-padded catalog id).

All 7 modalities share identical image_id sets per shard, zipped 1:1 by the standard 4M loader.

10.4 AbstractTransform (in the 4M training wrapper)

The `AbstractTransform` at 4M training time:
1. Reads the stacked `(n_trials, 17)` array for image_id.
2. Samples ONE random trial out of n_trials (uniform, varies per epoch).
3. If the mask is 0 (sentinel), uses 4M's standard masking mechanism (the eeg tokens are masked out, model can't see them, predicts other modalities).

Result: 4M sees one EEG trial per image per training step, drawn from the available pool. Across epochs the model effectively integrates across trials.

---

11. The §5 evaluation harness (in `neural_tokenizers/`)

11.1 The Tokenizer protocol

Defined at `neural_tokenizers/evaluation/protocol.py`. Any tokenizer must implement:

```python
class Tokenizer(Protocol):
    codebook_size: int
    def tokenize(self, x: Tensor) -> LongTensor: ...       # (B, C, T) -> (B, ...)
    def decode_tokens(self, tokens: LongTensor) -> Tensor: ...  # (B, ...) -> (B, C, T)
    # optional: tokens_to_embedding(tokens) -> (B, L, D)
```

LaBraMTokenizer satisfies this. It does NOT implement `tokens_to_embedding`, which means the probe uses bag-of-codes histograms instead of codebook-embedding mean-pool.

11.2 Four evaluation axes

1. **Reconstruction fidelity** — time MSE, per-channel Pearson r, spectral MSE (PSD via Welch). Spectral MSE is the diagnostic one for neural data.
2. **Codebook utilization** — perplexity = exp(H(p)), codes-used count, dead-code fraction.
3. **Downstream linear probe** — train a logistic regression on token features (bag-of-codes for us) to predict THINGS class. Report top-1 / top-5 on a held-out 20% split. Reference probes: raw flattened signal (upper bound) and uniformly random tokens (lower bound).
4. **Token-sequence statistics** — unigram entropy, bigram conditional entropy, run-length distribution. Pass criterion: bigram gap small (<20% of unigram), no degenerate marginals, mean run length near 1.

11.3 Why linear probe specifically

The rationale (per `neural_tokenizers/CLAUDE.md` §5.3): "if a linear model can read the stimulus off the tokens, the information is linearly decodable, which is what 4M's transformer needs from its input embeddings." A strong nonlinear probe would "overstate quality" by untangling things 4M wouldn't bother to.

Important caveat: 4M is not a linear model. It has a learned per-modality content embedding table and a transformer. It can absolutely learn nonlinear code-to-content relationships. A linear probe at chance is a strict lower bound on what 4M could extract, not an upper bound. So "probe at chance" does NOT mean "useless to 4M." It means "linearly unreadable." The actual usefulness is what the trunk team's scaling experiment measures.

11.4 Probe interface (useful technical detail)

`compute_probe_metrics(tokenizer, signal, labels, config, tokens=None)` accepts precomputed `tokens` as an optional argument. This is how the F3 A/B (§14.2) compares two tokenization recipes on the same signal/labels/config without subclassing the tokenizer.

12. §5 results (200-class identity probe — n=20k pilot)

NOTE: These are from an older evaluation using 200-class THINGS *image identity* (top-1 per image) on a 20,000-trial subsample. The canonical v2 results use the 27-class superordinate category probe on the full dataset. Both sets of numbers tell the same story (probe at random floor), but the v2 numbers are preferred for reporting.

Matched conditions: 10 EEG2 subjects, 20,000 trial seeded subsample, seed=0, 200-class THINGS image identity, probe_epochs=100, probe_test_frac=0.2.

metric	value	notes
Reconstruction		
time MSE	204.59	Not meaningful for this tokenizer (decoder reconstructs std-normalized spectra; unit mismatch).
channel Pearson r	0.148	Trustworthy scale-invariant metric.
spectral MSE (Welch)	1.39×10^9	Diagnostic for spectral preservation.
Codebook		
utilization	83.2%	6,814 of 8,192 codes used. Healthy.
dead-code fraction	16.8%	
perplexity	4,323	exp(H(p)).
entropy (nats)	8.37	Max possible 9.01.
Probe		**AT FLOOR**
top-1 tokens	**0.55%**	vs 0.67% random floor.
top-5 tokens	2.72%	vs 2.85% random floor.
top-1 raw signal	3.25%	The ceiling for linear decodability from this data.
top-5 raw signal	11.9%	
Sequence		
unigram entropy	8.37	
bigram conditional entropy	4.27	
entropy gap	49.0%	Mild warning (neighboring channels correlated).
mean run length	1.005	No collapse.
frac runs ≥ 2	0.080	

Three of four axes are healthy. The probe is at the random floor. The Stage-1 baseline (BottleneckMLP) was worse on every axis except possibly the probe (also at floor).

13. Code review (2026-05-28)

13.1 Scope

A two-pass code review was performed on all uncommitted EEG work: `.claude/`, `CLAUDE.md`, `.gitignore`, `modal/download_things_eeg.py`, `neural_tokenizers/eeg/` (entire subdir, all the new EEG tokenizer modules + Modal scripts), modifications to `modal/modal_app.py`, `modal/modal_demo.py`, `neural_tokenizers/CLAUDE.md`. The committed range against `origin/main` was empty (local main == origin/main at commit `87ea472` "MEG Tokenization"); the real change set was the uncommitted working tree.

13.2 Findings F1–F9

F1 — Superseded `produce_tokens.py` + `run_pipeline.py` would clobber the unified shards if run.
- `produce_tokens.py`: an old draft that averaged EEG across reps AND across subjects, used a private seed-20200220 80/20 split, wrote 6-digit positional stems instead of 9-digit catalog ids.
- `run_pipeline.py`: orchestrator that imported and ran this old draft, plus the EEG2-only 20-epoch finetune.
- Risk: `modal run run_pipeline.py` (or `produce_tokens.py`) would overwrite `/project/data/{train,val}/things/tok_eeg/` with wrong-stemmed averaged tokens that can't pair with the RGB shards.
- **Fix**: deleted both files. Grep confirmed no other importers.

F2 — EEG2 production tokenization skips the bandpass+notch filter that training applied.
- Training-time conversion filters 0.1–75 + 50 notch. Production-time `_extract_eeg2_trials` did not. EEG1 production path DID filter. Asymmetric and undocumented.
- **Fix**: added the filter to `_extract_eeg2_trials` with a loud DO-NOT-RE-RUN comment block. Did NOT re-run, did NOT regenerate shipped tokens. Whether to re-ship evaluated via A/B (see §14.3 — A/B says no).

F3 — Training window = 8 distinct trials; inference window = 8 identical copies.
- Documented as a verify-then-decide concern.
- **Tested empirically** (§14.2). Result: codes flip ~half, probe doesn't move. Not actionable.

F4 — Production load uses `strict=False`, hiding key mismatches.
- `model.load_state_dict(state_dict, strict=False)` silently ignores missing/unexpected keys. If checkpoint drifts, model loads random weights and emits garbage tokens with no error.
- **Fix**: capture missing/unexpected, assert no `quantize.*` or encoder keys are missing, raise loudly otherwise. Print count of non-load-bearing missing for visibility.

F5 — Stale docstrings (20 epochs / input_size 200) don't match shipped reality (10 epochs / input_size 1600).
- `run_full` and `run_full_combined` docstrings said 20 epochs; actual code is `epochs = 10`. CLAUDE.md prescribed `--input_size 200`; shipped is 1600 due to decoder bug.
- **Fix**: corrected docstrings; added an explicit "WHAT SHIPPED" note to the CLAUDE.md "Patch / segment configuration" section citing `finetune_labram_tokenizer.py:118-123,313` and `labram_tokenizer.py:78-85`.

F6 — Latent `NameError` in the shard packer's val cross-modal check.
- `modal_eeg_pack_shards.py:_print_cross_modal_alignment` referenced undefined `VAL_SHARD_DIR`. Currently unreachable (only called with `split="train"`, ternary lazily skips else branch), but a trap.
- **Fix**: added a DO-NOT-RUN banner at the top of `modal_eeg_pack_shards.py` (it's superseded by the unified repack), and pointed the `VAL_SHARD_DIR` reference at `VAL_LOOSE_DIR`. The packer's `_clean_old_outputs` does an `rmtree` on the live `tok_eeg/` dirs, so running it would delete the unified shards.

F7 — Token cache path mismatch in docstrings.
- Docstrings in `modal_eeg_produce_tokens.py` and `modal_eeg_pack_shards.py` say `labram_8192_ckpt5/`, but the actual constant is `labram/{SLUG}/`.
- **Fix**: corrected docstrings to `labram/V8192_d64_ch17_sr200_train-eeg1+2_e5/`.

F8 — No verifier exists for the unified repack.
- The immediate next task (verify the unified repack against the `.npz` caches) had no harness.
- **Fix**: wrote `verify_unified_shards.py` with `inspect` and `verify` entrypoints. Layout-agnostic (handles both stacked-per-image and one-file-per-trial). Read-only. See §15.

F9 — `decode_tokens` uses normalized "amplitude" as an FFT magnitude (technically wrong, but bounded by known caveat).
- LaBraM's `rec_amp` is std-normalized (can be negative), used directly as `r * exp(i*phase)` before `irfft`. Time-domain reconstruction is a qualitative inverse only.
- Bounded by the insight: "§5 reconstruction MSE is not meaningful... channel_corr is the only trustworthy reconstruction metric."
- **Fix**: added a NOTE comment in `decode_tokens` so the next reader doesn't "fix" the magnitude and then trust the MSE.

13.3 Files changed in the review

Deleted:
- `neural_tokenizers/eeg/modal/produce_tokens.py`
- `neural_tokenizers/eeg/modal/run_pipeline.py`
- (Plus their `.pyc` artifacts.)

Edited:
- `neural_tokenizers/eeg/labram_tokenizer.py` — F4 (load guard), F9 (NOTE comment), corrected the false "all 8 patch codes identical" claim with the measured 80% / 93% numbers (post-F8 patch diagnostic)
- `neural_tokenizers/eeg/modal/modal_eeg_produce_tokens.py` — F2 (added filter), F7 (docstring path).
- `neural_tokenizers/eeg/modal/modal_eeg_pack_shards.py` — F6 (banner + NameError fix), F7 (docstring path).
- `neural_tokenizers/eeg/modal/finetune_labram_tokenizer.py` — F5 (docstrings 20→10 epochs).
- `neural_tokenizers/eeg/modal/eval_labram_tokenizer.py` — added `--apply_filter` flag + `_FilteredTokenizer` wrapper for the F2 A/B.
- `CLAUDE.md` (root) — F5 (the "WHAT SHIPPED" note in Patch / segment configuration).

Added:
- `neural_tokenizers/eeg/modal/verify_unified_shards.py` — F8 verifier.
- `neural_tokenizers/eeg/modal/diagnose_patch_codes.py` — patch identity diagnostic (§14.1).
- `neural_tokenizers/eeg/modal/diagnose_f3_window.py` — F3 window A/B (§14.2).
- `.claude/memory/PROJECT_HANDOFF.md` — this document.

---

14. A/B experiments (2026-05-28)

14.1 Patch-code identity diagnostic (the collapse question)

Question: are LaBraM's 8 per-channel codes identical for tiled-identical input? The wrapper code originally claimed yes; measurement is the only way to know.

Script: `neural_tokenizers/eeg/modal/diagnose_patch_codes.py`. 512 sub-01 test trials, calibrated, tokenized via `model.get_codebook_indices` on tiled-identical (B, 17, 8, 200) input. Reshape output to (B, 17, 8) and check whether all 8 are equal per (trial, channel).

Sanity check: confirmed channel-major ordering by asserting `g[:, :, 0] == tokens_all[:, ::8]`.

Result (n = 512 × 17 = 8,704 channel-trials):
- All 8 patches identical: **80.2%**
- Mean distinct codes among 8: **1.22**
- Max distinct codes seen: 5
- Distribution: {1 distinct: 6,980, 2 distinct: 1,522, 3: 180, 4: 21, 5: 1}
- Patch 0 == majority of 8: **92.8%**

Verdict: the `[:, ::8]` collapse is NOT lossless dedup. The cause is LaBraM's per-patch `time_embed` making the same content encode differently per slot. But patch 0 is a stable representative (matches majority 93% of the time).

Code update: the comment in `labram_tokenizer.py:78-85` was corrected. The previous claim "all 8 patch codes are identical for tiled input" was false; the corrected comment cites the measurement.

14.2 F3 window context A/B (8-copy vs 8-real-trial)

Question: does the train/inference window-context mismatch cost downstream signal? At training, the 8 patches were 8 distinct consecutive real trials; at inference, they're 8 identical copies. The patch-identity result proves the codes are context-sensitive. How much does this matter?

Script: `neural_tokenizers/eeg/modal/diagnose_f3_window.py`. 8,000 sub-01 test trials, calibrated, tokenized two ways:
- **8-copy** (production): tile single trial 8×, encode, take patch 0 → (N, 17).
- **8-real** (training-style): for target trial i, build window from sig200[i:i+8] (with wrap-around), encode, take patch 0 → (N, 17). The 8 patches are 8 consecutive (random-order due to permutation) real trials.

Both pass through the same harness probe (`compute_probe_metrics` accepts precomputed tokens). Same signal, same labels, same config — only the tokenization recipe differs.

Result:

arm	top-1	top-5
8-copy (production)	0.25%	2.06%

arm	top-1	top-5
8-real (training-style)	0.38%	1.87%
raw ceiling	5.25%	15.9%
random floor	0.56%	2.38%

Plus: **code agreement between the two windows = 51.3%.** 8.27 of 17 channels flip per trial. 99.99% of trials change at least one channel.

Verdict: the window context flips half the codes but the probe doesn't move (both at random floor). F3 is real but not actionable. Re-tokenizing with training-style windows would not recover any object signal.

Note: this run is sub-01 only with n=8k, so the absolute probe numbers are noisier than the 10-subject 20k baseline. The relative comparison (8-copy vs 8-real) is the valid output, and it's a clear null.

14.3 F2 filter A/B (filtered vs unfiltered eval)

Question: does adding the missing bandpass+notch filter (matching training) at production time recover any downstream signal?

Setup: added `--apply_filter` flag to `eval_labram_tokenizer.py` with a `_FilteredTokenizer` wrapper that applies the F2-corrected path (upsample 100→200, then 0.1–75 bandpass + 50 notch at 200 Hz) before encoding. Defaults to off = current shipped behavior. Ran the harness both ways at matched conditions (10 subj, 20k seeded subset).

Result:

metric	unfiltered (shipped)	F2-filtered	verdict
probe top-1 (tokens)	0.55%	0.60%	noise (floor 0.67%)
probe top-5 (tokens)	2.72%	2.67%	noise (floor 2.85%)
codebook utilization	83.2%	82.8%	same
dead-code fraction	16.8%	17.2%	same
bigram entropy gap	49.0%	49.2%	same
channel_corr	0.148	0.148	same
mean run length	1.005	1.005	same
recon MSE	204.59	204.59	identical

Verdict: no meaningful difference on any axis. **Decision: do not re-ship.** Shipped tokens stay; the F2 code fix is in place so any future token run matches training.

Modal infra note: this A/B required `--detach` mode. A bare `modal run` client got terminated mid-stream and canceled the ephemeral app. The first detached attempt was killed by a Modal worker preemption ("Runner interrupted due to worker preemption"); the second detached attempt succeeded.

14.4 Synthesis across the three A/Bs

All three inference-side knobs (collapse, window context, preprocessing filter) are null on the probe. Combined with the earlier trial-averaging diagnostic (cleaner input makes the tokenizer WORSE), the conclusion is unambiguous:

The EEG token probe near-chance is intrinsic to the tokenizer representation, not to any inference-side recipe. A single 8,192-vocab code per channel per second is too coarse to carry the brief evoked stimulus signal under single-trial noise. Only a different tokenizer (V2: trial-averaged training, or non-LaBraM architecture, or both) would move the probe meaningfully. None of the inference choices can recover what isn't in the codebook.

---

15. Unified shard repack: independent verification

15.1 The verifier: `verify_unified_shards.py`

Read-only Modal job with two entrypoints:
- `inspect_main`: discover the on-disk layout — sample tar member names, .npy shapes, mask layout, coverage json, split json, cache structure.
- `main` (verify): full contract check.

The verifier is **layout-agnostic** (handles both stacked-per-image and one-file-per-trial). It hard-fails only on data-integrity invariants (data is real, not fabricated; coverage is consistent; no train/val leak). It reports membership alignment vs `things_split.json` and `eeg_coverage.json` separately because the authoritative shard universe is itself part of what's being checked.

15.2 What inspect found

Exact layout (empirically confirmed):
- 27 train + 27 val shard tars per split.
- Each shard owns `image_ids` in a 1000-id catalog range. Train shard_000 has 881 members; val shard_000 has 119; sum = 1000 = full range.
- Tar member: `<image_id>.npy`, 9-digit zero-padded.
- tok_eeg member shape: `(n_trials, 17) int16`, n_trials varies (e.g., 89, 90).
- eeg_mask member shape: `(1,) uint8`, value 0 or 1.

15.3 Verify result: clean PASS

```
[train]
  image_ids_total             22763
  nonsentinel                 19126
  sentinel                    3637
  trial_rows_total            1709455
  max_members_per_image       1
  ...
[val]
  image_ids_total             3344
  nonsentinel                 3344
  sentinel                    0
```

```
  trial_rows_total            329857
  ...
[_global]
  total_nonsentinel_images    22470
  coverage_union              22470
  nonsentinel_minus_union     0
  union_minus_nonsentinel     0

Hard-integrity violations: 0
RESULT: PASS — unified shards are consistent with the token caches.
```

Numbers reconcile end to end:
- Train 22,763 = `things_split.json` train count exactly.
- Val 3,344 = `things_split.json` val count exactly (and all non-sentinel, because val is the strict catalog ∩ MEG ∩ EEG1 ∩ EEG2 intersection — every val image has EEG).
- 22,470 non-sentinel total = `eeg_coverage.json` union exactly.
- 3,637 train sentinels = `eeg_coverage.json` n_uncovered exactly.
- 1,709,455 + 329,857 = **2,039,312** trial rows = source `.npz` cache total exactly. Every single cached row made it into the shards.
- `max_members_per_image = 1` confirms stacked layout (one .npy per image, not per trial).

Conclusion: the unified repack is byte-faithful to the source caches. The "was the repack correct?" open question is closed.

---

16. Implications for 4M

Three layers, kept honest:

16.1 The tokens deliver to 4M as designed

17 tokens per trial, vocab 8,192, integer codes, per-image stacked layout matched to RGB shards via image_id, mask folder for missing-EEG images, AbstractTransform for random-trial sampling per epoch. The trunk lead can ingest today.

16.2 Per-image linear stimulus decodability from these tokens is at chance

This is a real property of the shipped tokenizer at single-trial granularity. We empirically ruled out collapse, window context, and preprocessing filter as causes. The remaining cause is the tokenizer representation itself (single-trial LaBraM at one code per channel per second), and the trial-averaging diagnostic confirms it (tokenizer collapses on cleaner input).

It's NOT fixable by tweaking the inference recipe. It would require V2 (different tokenizer).

16.3 The probe at chance does not directly answer the scaling question

The §5 probe is intentionally linear. 4M's transformer is nonlinear and has a learned per-modality content embedding table that can absorb nonlinear code-to-content relationships. "Tokens are not linearly decodable" is a strict lower bound on what 4M can use, not an upper bound. The actual scaling question is what the trunk team's training experiment measures.

16.4 Preliminary scaling result

The trunk team's preliminary result (2026-05-28): the biggest model (50M params) performs worse than a smaller model when MEG is included. The biggest models also perform poorly with EEG — at larger models, the network appears to overfit to neural noise.

This is **consistent** with the probe finding, not in conflict with it. If neural tokens are effectively noise for stimulus discrimination, larger models have more capacity to fit that noise and degrade more on real targets. The two observations tell a coherent story:

- Tokens carry minimal linearly-decodable per-image identity.
- Larger 4M models overfit to whatever is in those tokens.
- The 50M-with-neural model is worse than 5M-with-neural because the bigger model fits the EEG noise harder.

Open question: **what does 50M look like WITHOUT neural data?** The "biggest model worse with neural" story is only diagnostic of the EEG/MEG modality if the no-neural 50M baseline is fine. If 50M-no-neural is also worse than 5M-no-neural, it's a generic scaling pathology unrelated to neural data.

---

17. Team track ownership and design decisions

17.1 EEG track

The EEG track owns: THINGS-EEG1 and THINGS-EEG2 downloads and preprocessing, the LaBraM finetune pipeline, the per-subject `.npz` token caches, and the harness wrapper (`LaBraMTokenizer`).

Key decisions:
- Image-level 80/20 split (seed 20200220) for tokenizer training HDF5. Overridden by the team's `things_split.json` for 4M training.
- Per-channel µV calibration: rescale EEG2 channels to match median EEG1 per-channel std (15.25 µV). Median chosen over mean due to ~18/50 artifact-contaminated EEG1 subjects.
- Trials kept separate (not averaged): gives 4M more samples, consistent with the MEG per-subject convention.
- Checkpoint selection: epoch 5 of 10 (val loss minimum), not final epoch.

17.2 MEG/infra track

The MEG/infra track owns: THINGS-MEG downloads, the BrainOmni MEG tokenizer pipeline, the eval harness (`neural_tokenizers/`), the unified shard packing, and the AbstractTransform in the 4M training wrapper.

Eval harness contract (`neural_tokenizers/CLAUDE.md` §5): defined the Tokenizer protocol, the four §5 axes, the modality-agnostic harness structure. Set the linear-probe-as-gate convention. Set the "two-stage" structure (Stage 1 baseline before Stage 2 production).

Channel-count note (open, low priority): `neural_tokenizers/CLAUDE.md` originally assumed THINGS-EEG2 is `(63, 100)` per trial. Verified from `gifale95/eeg_encoding_model/02_eeg_preprocessing/preprocessing_utils.py`: the published preprocessed version selects only 17 channels via regex `^O *|^P *`. The eval-harness probe should not assume 63 channels.

Unified format design (2026-05-26 team decision): 4M's `unified_datasets.py` zips image-aligned modalities row-by-row 1:1, which doesn't natively handle multi-trial neural data. Solution: stacked `.npy` per image_id + sentinel + mask folders + AbstractTransform for random trial selection. No 4M edits required.

Question raised about token format (answered with measurement from §14.1): the 17-token output is collapsed from the native 136 codes (17ch × 8 time-patches) via `[:, ::8]`. This is a stable representative pick: 80% of channel-trials have all 8 patches identical, patch 0 is the majority code in 93% of cases. NOT lossless dedup — LaBraM's per-patch `time_embed` makes identical content encode differently per slot.

17.3 Trunk lead

The trunk lead owns: mini-4M architecture, CC12M RGB crawler, Depth Anything pseudo-labeling pipeline, and the overall 4M training runs that produce the scaling results.

RGB shards live at `/project/data/{train,val}/things/rgb/shard_NNN.tar`. The 27-shard 1000-image catalog-range convention originates from the trunk's shard plan.

Design preference: let the model learn trial averaging itself (random 1-of-n-trials per epoch via AbstractTransform) rather than forcing the tokenizer or dataloader to average.

Preliminary scaling result: largest models (50M) perform worse with neural data than without it, consistent with overfitting to token noise.

Open item: single-subject sweep proposal (use existing tokens, subset to one subject for 4M training, no re-finetune needed since the tokenizer is fixed).

17.4 Cross-cutting decisions

- **2026-05-22, scope narrowing**: drop bidirectional EEG↔MEG translation; drop fMRI angle. Focus on the scalability question with EEG.
- **2026-05-22, image-level 80/20 split**: per modality, then later unified to `things_split.json`.
- **2026-05-22, tokenization paradigm split**: MEG track uses non-trained μ-transform (Cho2026); EEG track uses trained VQ-VAE (LaBraM finetune). Both produce discrete int tokens with the same `tokenize() / decode_tokens() / codebook_size` interface.
- **2026-05-22, Modal wrapper organization**: modality-specific Modal wrappers live in `neural_tokenizers/<modality>/modal/`, not in the top-level `modal/`.
- **2026-05-26, val split = strict intersection**: catalog ∩ MEG ∩ EEG1 ∩ EEG2 = 3,344 images. Every val image has all four modalities.
- **2026-05-26, unified format**: stacked per-image .npy + sentinel + mask folders + AbstractTransform for random trial sampling.
- **2026-05-28, F2 not re-shipped**: empirical eval A/B null. Shipped tokens stay.
- **2026-05-28, single-subject sweep uses existing tokens**: no re-finetune. Tokenizer is fixed; subsetting is a 4M-side operation.

18. Files changed 2026-05-28 — full list

18.1 Edited (source-only, no volume writes)

Path	What changed	Why
`neural_tokenizers/eeg/labram_tokenizer.py`	Added load-state-dict guard (F4); added NOTE comment on `decode_tokens` reconstruction caveat (F9); corrected the false "all 8 patch codes identical" claim with measured 80% / 93% numbers	Hardening + accuracy of inline docs
`neural_tokenizers/eeg/modal/modal_eeg_produce_tokens.py`	Added bandpass+notch filter to `_extract_eeg2_trials` (F2) with a loud DO-NOT-RE-RUN comment; corrected docstring cache path (F7)	Match training-side preprocessing, prevent silent re-shipping
`neural_tokenizers/eeg/modal/modal_eeg_pack_shards.py`	Added do-not-run banner at the top (F6); fixed `VAL_SHARD_DIR` NameError (F6); corrected docstring cache path (F7)	This packer is superseded by the unified repack and would `rmtree` the live `tok_eeg/` dirs if run
`neural_tokenizers/eeg/modal/finetune_labram_tokenizer.py`	Corrected docstrings 20→10 epochs (F5)	Match shipped reality
`neural_tokenizers/eeg/modal/eval_labram_tokenizer.py`	Added `--apply_filter` flag + `_FilteredTokenizer` wrapper class	Enable F2 A/B without touching shipped behavior (flag defaults to off)
`CLAUDE.md` (root)	Added "WHAT SHIPPED" override note to Patch / segment configuration section (F5)	Document the `input_size=1600` tiling workaround that overrides the spec
`.claude/memory/state.md`	Updated Current state + Last session + Decisions + Open questions + Insights	Session log

18.2 Added

Path	What it is
`neural_tokenizers/eeg/modal/verify_unified_shards.py`	Read-only verifier for the unified repack. Layout-agnostic. Two entrypoints: `inspect_main` (discover) and `main` (verify). Result this session: clean PASS.
`neural_tokenizers/eeg/modal/diagnose_patch_codes.py`	Diagnostic: measure whether LaBraM's 8 per-channel codes are identical for tiled-identical input. Result: 80% identical, patch 0 = majority 93%.
`neural_tokenizers/eeg/modal/diagnose_f3_window.py`	F3 A/B: 8-copy window (production) vs 8-real-trial window (training-style). Same target trial, same patch 0 read, different context. Result: codes flip ~50%, probe unchanged.
`.claude/memory/PROJECT_HANDOFF.md`	This document.

18.3 Deleted

Path	Why
`neural_tokenizers/eeg/modal/produce_tokens.py`	Superseded; averaged across reps + subjects, used wrong split, used wrong stems. Would clobber `tok_eeg/` if run.

Path	Why
`neural_tokenizers/eeg/modal/run_pipeline.py`	Orchestrator that imported `produce_tokens.py`. Same risk.

Confirmed via grep that nothing else imported either file.

19. Active scripts inventory

19.1 `neural_tokenizers/eeg/modal/` — Modal wrappers

Script	Purpose	Status
`_app.py`	Shared Modal `App("neural-fm")` + `Volume.from_name("project")` reference. All EEG modal scripts import from here.	Active
`convert_eeg_to_labram_hdf5.py`	EEG2 .npy → LaBraM HDF5. Reads `preprocessed_eeg_{training,test}.npy`, calibrates per-channel, upsamples 100→200, filters 0.1–75 + notch 50, concatenates trials end-to-end, writes HDF5 per subject.	Active (was used for shipped training)
`convert_eeg1_to_labram_hdf5.py`	EEG1 .set → LaBraM HDF5. Reads EEGLAB, parses BIDS event TSVs for E1 onsets, epochs −0.2 to +0.8 s, filters, resamples 250→200, writes HDF5. Also has a `compute_channel_stats` function that produced the `eeg1_per_channel_std.json` calibration target.	Active (was used for shipped training)
`finetune_labram_tokenizer.py`	The Modal wrapper for LaBraM `run_vqnsp_training.py`. Strips optimizer+scaler from `vqnsp.pth`, patches the HDF5 paths into a launch-time copy of `run_vqnsp_training.py`, launches via `torchrun --nproc_per_node=1`. Local entrypoints: `finetune_standalone` (dry run), `run_full` (EEG2 only, 10 epochs), `run_full_combined` (EEG1+EEG2, 10 epochs — the SHIPPED recipe).	Active
`modal_eeg_produce_tokens.py`	Per-subject token production. For each (source, subject), loads raw data, calibrates, tokenizes via the LaBraM wrapper, writes `<source>_<subject>.npz` to `/project/data/things-eeg/tokens/labram/<slug>/`. Includes startup calibration assertion. Has F2 filter fix in `_extract_eeg2_trials` but not re-run.	Active (was used for shipped tokens)
`modal_eeg_pack_shards.py`	Original 23-shard per-trial packer. **DO NOT RUN** — superseded by the unified 27-shard repack; its `_clean_old_outputs` would `rmtree` the live `tok_eeg/` dirs. Kept for reference.	Superseded
`modal_eeg_reorganize_volume.py`	Reorganized the volume layout. One-shot script that already ran.	Already ran
`modal_eeg_write_coverage_json.py`	Writes `/project/data/eeg_coverage.json` mapping THINGS catalog image_ids to EEG1/EEG2 coverage sets. Already ran.	Already ran
`check_image_coverage.py`	Sanity-check for image_id mapping between EEG1 events and EEG2 metadata and the THINGS catalog.	Active diagnostic
`download_eeg2_image_metadata.py`	Pulls `image_metadata.npy` from OSF for EEG2 (the filename ↔ image-index mapping).	Already ran
`inspect_vqnsp_checkpoint.py`	Diagnostic that dumps the structure of `vqnsp.pth` (keys, shapes, saved args, codebook health). Use before launching a training run if anything seems off.	Active diagnostic
`eval_labram_tokenizer.py`	Runs the §5 four-axis harness on a LaBraM checkpoint. Has `--apply_filter` flag added this session.	Active
`run_eval_harness.py`	Older eval entrypoint. Possibly superseded by `eval_labram_tokenizer.py` (similar functionality). Check before using.	Active (verify scope)
`train_stage1.py`	Trains the Stage-1 BottleneckMLP baseline. 20-epoch CPU/GPU job. Already ran.	Already ran
`verify_unified_shards.py`	Read-only verifier for the unified repack. `inspect_main` then `main`.	Active
`diagnose_patch_codes.py`	Measures the patch-identity question.	Active
`diagnose_f3_window.py`	F3 window context A/B.	Active

19.2 `neural_tokenizers/eeg/` — Python library code

Module	Purpose
`__init__.py`	Re-exports `Stage1EEGTokenizer`
`labram_tokenizer.py`	`LaBraMTokenizer` class: wraps the LaBraM model + codebook in the harness `Tokenizer` protocol. Implements `tokenize` (with the 8-tile + patch-0 collapse), `decode_tokens` (with the std-normalized-amplitude caveat in §6). Has the hardened `_load_vqnsp` with strict-load guard.
`eeg_encoder.py`	`BottleneckMLPEncoder` for Stage 1 baseline.
`eeg_decoder.py`	`BottleneckMLPDecoder` for Stage 1 baseline.
`eeg_quantizer.py`	`ProductVQQuantizer` (8 codebooks × 1024, straight-through gradient) for Stage 1 baseline.
`eeg_tokenizer.py`	`Stage1EEGTokenizer` composing the three Stage 1 modules.

19.3 `modal/` — shared scaffolding (top-level)

Script	Purpose
`modal_app.py`	Shared `App("neural-fm")` + `Volume("project")` reference at the top level. EEG modal scripts duplicate this in their own `_app.py`.
`modal_demo.py`, `demo_train.py`	Template for the "two-file pattern" (plain Python training script + thin Modal wrapper).
`MODAL_GUIDE.md`	Team-facing primer on Modal usage, secrets, paths.
`download_things_eeg.py`	Downloads EEG1 and EEG2 raw data to the volume. Already ran for both.
`things_manifest.py`, `modal_things_repack.py`	RGB shard packer (split + repack).
`meg_token_shard.py`, `modal_meg_pack_shards.py`	MEG shard packer.
`modal_meg_pipeline_audit.py`	28-check end-to-end audit for the MEG cache + shards. Pattern for our verifier.

19.4 External submodules

- `external/ml-4m/` — `apple/ml-4m`. NOT a fork. To modify 4M, need to fork and update submodule URL.
- `external/LaBraM/` — `935963004/LaBraM`. Added 2026-05-22. We do not modify; we patch at launch time via the Modal wrapper.
- `external/Cho2026_Tokenizer/` — Cho 2026 paper reference code (MEG track reference).

20. Decisions log (chronological, all dates 2026)

- **05-22**: Scope narrowed to scalability research question. EEG↔MEG translation and fMRI dropped.
- **05-22**: Use Gifford's preprocessed THINGS-EEG2 as-is for downloads; defer MVNN-vs-µV resolution to conversion time.
- **05-22**: Image-level 80/20 train/val split.
- **05-22**: Modal volume name is `project`. App is `neural-fm`. Workspace is `neural-fm-data`.
- **05-22**: HDF5 is the format for LaBraM Stage-1 input.
- **05-22**: MEG-GPT ruled out for THINGS-MEG (no Polhemus head shapes).
- **05-22**: Tokenizer interface = `tokenize() / decode_tokens() / codebook_size`. Stage 1 baseline before any Stage 2.
- **05-22**: Data layout on volume — tokenized at `/project/data/[train,val]/[dataset]/tok_[modality]`, raw at `/project/data/raw/[dataset]/`.
- **05-22**: No cross-validation. Single 80/20 image-level split.
- **05-22**: NSD may come back for brain-alignment eval only.
- **05-22**: Train tokenized output = WebDataset .tar shards, ~5000 images per shard.
- **05-22**: One tokenized entry per unique image (later overridden by per-trial / per-rep, then by the unified format).
- **05-22**: Val tokenized output = flat .npy in `class_0/` (later overridden by unified shard format).
- **05-22**: All Modal scripts mount `project` at `/project`.
- **05-22**: MEG and EEG use different tokenization paradigms. MEG = non-trained µ-law (Cho2026). EEG = trained VQ-VAE (LaBraM finetune).
- **05-22**: Modal wrapper organization — modality-specific wrappers live in `neural_tokenizers/<modality>/modal/`.
- **05-23**: LaBraM Stage-1 spec verified end-to-end. HDF5 continuous (not epoched), 200 Hz at write time, uppercase channel names in `standard_1020`, `lFreq/hFreq/rsFreq/chOrder` attrs, no `-data_path` flag (patch in wrapper), `torchrun` required even on single GPU, `-codebook_emd_dim 64`, strip optimizer+scaler from `vqnsp.pth` before `-resume`.
- **05-23**: Default MVNN resolution path = run with MVNN-whitened data unchanged. Escalation: empirical rescale, then raw re-preprocess. (Later overridden in practice — codebook collapsed on MVNN as-is, switched to per-channel µV calibration.)
- **05-23**: First Modal training run = `-epochs 1` on 1 subject for smoke test.
- **05-25**: EEG2 µV calibration = per-subject std rescaled to fixed EEG1 median per-channel std vector. Median (not mean) pooling — ~18/50 EEG1 subjects have artifact-contaminated channels.
- **05-25**: Ship checkpoint-5 (10-epoch EEG2+EEG1 combined) as production EEG tokenizer.
- **05-25**: Token production keeps trials SEPARATE (not averaged), capped at 5 per (image, source). Gives 4M more samples.
- **05-25**: EEG1 and EEG2 combined into one `tok_eeg/` dataset; source in filename suffix.
- **05-26**: Val split = `things_split.json` val_image_ids (3,344, strict intersection).
- **05-26**: Volume layout mirrors `things-meg/`. Slug = `V8192_d64_ch17_sr200_train-eeg1+2_e5`. config.json carries calibration + ckpt SHA256.
- **05-26**: Team adopted UNIFIED single-dataset structure. All 7 modalities in one dataset zipped 1:1 by standard 4M loader. One .npy per image_id with trials stacked. Sentinels + uint8 mask folders for missing neural data. 27 catalog-range shards. Random trial selection + masking in train-script AbstractTransform.
- **05-26**: Team preference: let the model learn trial averaging itself rather than forcing it in preprocessing.
- **05-26**: THINGS val (3.3k) may be skippable since scaling is evaluated on RGB/depth (+ cc12m), not neural. Keeping for now.
- **05-28**: Do NOT re-ship EEG2 tokens with F2 filter. Eval A/B shows probe 0.55% vs 0.60% (within noise; random 0.67%), util 83.2% vs 82.8%, every other axis identical. F2 code fix stays so future runs match training.
- **05-28**: F3 window-context mismatch real but not actionable. 8-real vs 8-copy flips ~49% of codes but probe unchanged at random floor.
- **05-28**: Collapse 136→17 is NOT lossless dedup. 80% of channel-trials all 8 identical, patch 0 = majority 93%. LaBraM's per-patch `time_embed` makes identical content encode differently by slot. Comment in `labram_tokenizer.py` corrected.
- **05-28**: Deleted `produce_tokens.py` + `run_pipeline.py`. Marked `modal_eeg_pack_shards.py` DO-NOT-RUN.
- **05-28**: `-apply_filter` flag added to `eval_labram_tokenizer.py` (additive, default off).
- **05-28**: Long Modal evals require `modal run --detach`. Even detached runs can die to worker preemption; recovery = retry.
- **05-28**: Unified repack VERIFIED clean. 0 hard-integrity violations. Every cache row accounted for.

21. Open questions / next steps

In rough priority order:

1. **Trial-to-trial tokenization variability.** Compute within-image vs cross-image modal-code agreement from the `.npz` caches. (4-48 trials/image for MEG, mean 4.48; EEG varies similarly.) Status: pending.
2. **Single-subject sweep.** Proposal: use existing tokens, subset to one subject for 4M training, no re-finetune needed. Open questions: (a) which subject — one EEG2 (82k trials, multi-rep) or one EEG1 (~22k, single-rep RSVP)? and (b) what hypothesis the sweep tests. Status: open.
3. **Coarser probe target.** Run probe on animacy (binary) or THINGS superordinate categories (~27) rather than 200-way image identity. Tells us whether tokens carry coarse semantic info. Requires `image_id → animacy / superordinate` mapping; may already exist under `meg/data.py` per `meg/CLAUDE.md` §5.3. Status: open.
4. **V2 EEG tokenizer (post-milestone).** The only remaining lever to move the probe off the random floor. Options:

- Train LaBraM on trial-averaged input. For EEG2 within-subject averaging (4 reps → √4 = 2× SNR). For EEG1 nothing to average within (image, subject) (RSVP single-rep). Asymmetric across datasets.
- Average across subjects too (√40–√50 SNR, mimics the 80-rep diagnostic's regime), but loses per-subject identity and conflicts with the 2026-05-25 "trials separate" decision.
- Different architecture (non-LaBraM, e.g., 1-D CNN over time with explicit spectral loss, or temporal-patch transformer trained from scratch on THINGS).
- **Caveat**: LaBraM pretrained weights may not transfer to averaged input (pretraining was on continuous resting-state). Smoke test on averaged data before committing.
- Status: post-milestone.

5. **AbstractTransform validation.** Does masking-for-missing-neural-data train cleanly in 4M? Validate on first training run.
6. **Channel-count note in** `neural_tokenizers/CLAUDE.md` (63 vs 17 for EEG2). Low priority.
7. **No-neural 50M baseline.** What does the no-neural 50M model look like at the same training budget? This is the comparison that distinguishes "neural is hurting" from "generic scaling pathology."

---

22. V2 tokenizer planning notes

The case for V2: every inference-side knob tested came back null on the probe. The trial-averaging diagnostic positively showed the tokenizer is overfit to single-trial noise (cleaner input → tokenizer collapses). So V2 means a different *trained* tokenizer, not a different deploy recipe.

22.1 The 4-rep averaging option (within-subject EEG2)

Replace EEG2 conversion's reshape with mean over axis=1:

```
# Current: (16540, 4, 17, 100) -> (16540*4, 17, 100). 4 rows per image.
# V2: (16540, 4, 17, 100).mean(axis=1) -> (16540, 17, 100). 1 row per image.
```

√4 = 2× SNR. Modest. The raw probe goes 3.25% → maybe 4-4.5%, not 6.5% (which was √80).

22.2 The EEG1 asymmetry (the gotcha)

EEG1 has 1 rep per (image, subject). RSVP shows each image once. **Nothing to average within (image, subject).** Three options:
- **(a) EEG2 averaged + EEG1 single-trial.** Mixed noise regime during training. The model adapts to variable input quality. Easiest to implement.
- **(b) Cross-subject averaging for both.** EEG2: 4 reps × 10 subj = 40 trials → √40 ≈ 6× SNR. EEG1: 50 subj × 1 rep = 50 trials → √50 ≈ 7× SNR. Mimics the diagnostic's regime. But loses per-subject identity — conflicts with the 2026-05-25 "trials separate" decision.
- **(c) Bootstrap subsets across subjects.** Pick N random subjects per image, multiple draws. Keeps some diversity, reduces noise meaningfully. Hybrid.

22.3 Deployment-time question (separate from training-time)

If V2 is trained on averaged data, what does the tokenizer see at deployment?
- **Deploy single-trial**: inverts the current mismatch. Tokenizer learned clean signal, given noisy at runtime. Probably bad.
- **Deploy averaged**: gives 4M one token vector per image instead of ~90. Way less training data but cleaner. Reverses the 2026-05-25 decision; requires team alignment before committing.

22.4 The pretrained-weights-transfer risk

LaBraM was pretrained on continuous resting-state EEG. Averaged THINGS is a DIFFERENT distribution shift than single-trial THINGS — not just cleaner, structurally different. The pretrained codebook may not transfer. Could be forced from Stage 2c (LaBraM weights + finetune) to Stage 2a (scratch design + scratch weights), which is a much bigger commitment. Smoke test = 1-epoch dry run on averaged data, watch `Unused_code`. If it spikes to ~8,192, weights don't transfer.

22.5 Validation reality check

Even with perfect averaging, the raw probe ceiling is 6.5% on 200-way (the 80-rep diagnostic). EEG just doesn't carry abundant per-image identity at the linear level. V2 would help, but wouldn't approach a 4M-RGB-token-quality ceiling.

A cheaper diagnostic before committing to V2: run the coarser-target probe (animacy, superordinate) on the CURRENT shipped tokens. Two outcomes:
- Pass coarse, fail fine → 200-way was too hard, tokenizer is OK, no V2 needed.
- Fail coarse → tokens really are noise, V2 needed.

This is several hours of work vs several days for V2.

---

23. Modal infrastructure

23.1 App + volume

- **App**: `neural-fm` (workspace `neural-fm-data`, the only workspace).
- **Volume**: `project`. Created without `create_if_missing`. Mounted at `/project` in containers.
- **Secrets**: workspace-scoped (visible to all workspace members). Personal HF tokens follow `hf-<initial>` naming (e.g., `hf-eeg`, `hf-trunk`, `hf-meg`). NOT shared via `modal_app.py`; each team member's secret is referenced by name where needed.

23.2 Authentication

`modal profile current` returns `neural-fm`. Auth is at `~/.modal.toml`.

23.3 `-detach` lesson (2026-05-28)

Bare `modal run` blocks while streaming logs from the ephemeral app to the client. If the client process is killed (terminal closed, session interrupted, etc.), the ephemeral app receives a cancellation signal and stops. **For long-running evals (anything that might exceed 10 minutes), always use `modal run --detach`.** The app then runs to completion independently of the client.

Caveat: `--detach` only keeps the *last triggered* function alive after the client disconnects. If a script triggers multiple functions, only the last one survives a disconnect.

23.4 Preemption recovery (2026-05-28)

Modal workers can be preempted ("Runner interrupted due to worker preemption. Your Function will be restarted with the same input."). On detached runs, preemption may cause cancellation if the client is already gone. Recovery: plain retry. Two distinct things to watch for in app logs:
- "Runner interrupted due to worker preemption" → Modal is reclaiming the worker; the function will restart.
- "Received a cancellation signal while processing input" → the run is canceling; the function will NOT restart unless re-launched.

23.5 Running Modal CLI from this environment

`modal` is at `/Users/lianeozoemelam/miniconda3/bin/modal`. Version 1.4.3. Profile is set.

Background-bash launches with `run_in_background=True` capture the modal CLI stdout to a file in `/tmp/`. The first line containing `ap-...` is the app ID, which can be passed to `modal app logs ap-XXX` to retrieve remote logs independent of whether the client survived. For long jobs always grab the app ID early via `grep -o "ap-[A-Za-z0-9]*" /tmp/<log>.log | head -1`.

23.6 Script gotcha: multiple local entrypoints

If a script has multiple `@app.local_entrypoint()` functions, plain `modal run script.py` errors out with "Specify a Modal Function or local entrypoint to run." Use `modal run script.py::<entrypoint>` explicitly. We have this in `verify_unified_shards.py` (`inspect_main`, `main`) and `eval_labram_tokenizer.py` (`main`) and `finetune_labram_tokenizer.py` (`finetune_standalone`, `run_full`, `run_full_combined`).

23.7 GPU choices used

- **L40S**: default for LaBraM finetune. ~24h runs supported.
- **A10G**: smaller, cheaper, good for fast inference jobs. Used for the F3 A/B diagnostic (~2 min on A10G).
- **A100**: not yet used.
- **CPU**: most eval and conversion jobs. The probe trains on CPU (~5 min for 100 epochs on 20k samples).

GPU memory has not been a constraint at our scale.

---

25. Glossary

- **4M**: Massively Multimodal Masked Modeling. Mizrahi et al. 2023, NeurIPS Spotlight. The foundation model architecture we're building on. Each modality is tokenized to discrete IDs; one shared transformer trained with cross-modality masked-token objective.
- **AbstractTransform**: 4M-side data transform applied at training step; wraps random trial selection + masking for missing neural data.
- **BIDS**: Brain Imaging Data Structure. Standard for neuroimaging data layout.
- **BrainOmni**: Pre-trained brain tokenizer from OpenTSLab. RVQ + SEANet. Used for MEG.
- **Cho2026**: Cho et al. 2026 EphysTokenizer paper. Source of the μ-transform baseline and the learned AE tokenizer (Phase 2 for MEG, not implemented).
- **codebook**: discrete dictionary of token embeddings. Vocab size = number of entries.
- **CTF**: a brand of MEG sensors (axial gradiometers). THINGS-MEG uses CTF.
- **EEGLAB**: MATLAB toolbox for EEG analysis. THINGS-EEG1 derivatives are in EEGLAB .set/.fdt format.
- **EMA**: exponential moving average. NormEMAVectorQuantizer updates codebook entries via EMA, not gradient descent.
- **F1–F9**: code review findings. See §13.2.
- **HDF5**: hierarchical data format. LaBraM's expected input file format.
- **LaBraM**: Large Brain Model. Jiang et al. 2024, ICLR. Pretrained EEG foundation model. We finetune its `vqnsp.pth` for our EEG tokenizer.
- **MEG-GPT**: a MEG foundation model project. Ruled out for our use because THINGS-MEG lacks Polhemus head digitization required for source reconstruction.
- **mini-4M**: the trunk lead's smaller 4M variant. Built from scratch. Architecturally close to Tiny (30M) / Small (75M) configs in `fourm/models/fm.py`.
- **modal**: cloud GPU/CPU compute platform we use for jobs. `https://modal.com`.
- **MVNN**: multivariate noise normalization. Per-channel z-score + cross-channel decorrelation. Gifford applies it as preprocessing on EEG2.
- **NeuralTransformer**: LaBraM's encoder architecture. 12-layer transformer with channel and time positional embeddings.
- **NormEMAVectorQuantizer**: LaBraM's quantizer. L2-normalized codebook, EMA-updated.
- **NSD**: Natural Scenes Dataset (fMRI). Dropped from project scope; possible eval-side later.
- **OpenNeuro**: neuroimaging data repository. Hosts THINGS-EEG1 as ds003825.
- **OSF**: Open Science Framework. Hosts THINGS-EEG2.
- **patch**: a 200-sample (= 1 second at 200 Hz) time block in LaBraM. The model operates on per-channel 1-second patches.
- **Polhemus**: brand of head-shape digitization device for MEG/EEG. THINGS-MEG lacks it; only 3-point fiducials.
- **probe**: a classifier trained on tokenizer outputs to predict stimulus identity. §5 uses linear (logistic regression on bag-of-codes).
- **RSVP**: rapid serial visual presentation. THINGS-EEG1 paradigm.
- **RVQ**: residual vector quantization. BrainOmni uses 4-layer RVQ for MEG tokens.

- **SEANet**: encoder-decoder backbone in BrainOmni.
- **§5**: section 5 of `neural_tokenizers/CLAUDE.md`, the four-axis eval contract.
- **shard**: a WebDataset .tar file containing many samples for streaming during training. Our shards are 1000-image catalog ranges.
- **Stage 1 / Stage 2 / 2a / 2b / 2c**: tokenizer design taxonomy. Stage 1 = simple baseline (BottleneckMLP + memcodes). Stage 2 = production with stronger backbone + spectral loss. 2a = scratch design + scratch weights. 2b = LaBraM arch + scratch weights. 2c = LaBraM arch + LaBraM weights, finetuned (what we ship).
- **std_norm**: LaBraM's spectral normalization step. Zero-mean unit-std across (N, A, T) for amplitude and phase separately.
- **superordinate categories**: THINGS coarse groupings (~27 categories from 1854 concepts). Prescribed as the MEG probe target.
- **THINGS**: object-image dataset with hierarchical labels. 26,107 images, 1,854 concepts, ~27 superordinate categories.
- **THINGS-EEG1**: Grootswagers et al. 2022. 50 subjects, RSVP, 1 rep/(image, subject).
- **THINGS-EEG2**: Gifford et al. 2022. 10 subjects, slow event-related, 4–80 reps/(image, subject).
- **THINGS-MEG**: Hebart et al. 2023. 4 subjects, 27,048 trials/subject. MEG track data.
- **tile**: replicate input across the time-patch axis. At inference we tile a 1-second trial 8× to match `input_size=1600`.
- **time_embed**: LaBraM's per-time-patch positional embedding. The cause of patch-code divergence (the patch-identity diagnostic finding).
- **time_window**: LaBraM hyperparameter. `time_window * 200 = window_size_samples`. We use `time_window=8`.
- **TVSD**: intracortical/Utah-array data. Future modality, not in current scope.
- **vqnsp**: LaBraM's "VQ Neural Spectrum Predictor", the Stage-1 tokenizer model class.
- **WebDataset**: PyTorch streaming dataset format using tar shards.

---

27. Quick-start checklist

If picking up cold:

1. **Read** `.claude/memory/state.md` for the latest session log entry.
2. **Read root** `CLAUDE.md` for project rules and decided plan.
3. **Read** `neural_tokenizers/CLAUDE.md` for the eval harness contract.
4. **Glance at this document's TL;DR (§0), Synthesis (§14.4), Implications (§16), Open questions (§21).**
5. **Confirm volume is reachable**: `modal profile current` should return `neural-fm`. `modal volume ls project | head` should show the layout.
6. **If launching anything**: prefer read-only first. Use `-detach` for jobs >10 min. Never run `produce_tokens.py` or `run_pipeline.py` (deleted) or `modal_eeg_pack_shards.py` (DO-NOT-RUN). The current shipped state is verified clean; do not regenerate without explicit clearance.
7. **Working style**: no em dashes, plain English, verify before stating, don't ask clarifying questions unless blocked, don't commit without explicit instruction.